

TP02 : Fonctions Et Paramètres

Dans un script python tp2.py écrire les exercices suivants :

Compréhension du code :

1. Appel de fonctions avec des arguments :

Remarque : Les arguments sont les valeurs données à la fonction lors de son appel.

1.1 Fonctions avec des paramètres

- Faire un appel des fonctions suivantes avec des arguments, récupérer la valeur renvoyée par chaque fonction et afficher la. Essayer les appels avec les 3 types d'arguments suivants: int, float et str.

a) `def f1(x):`
 `return x + 1`

b) `def f2(x, y):`
 `z = x + y`
 `return z`

- Qu'affiche le code suivant :

c) `def f3(x, y):`
 `return x - y`

 `res1 = f3(2, 3)`
 `res2 = f3(3, 2)`
 `print(res1, res2)`

1.2 Fonctions et paramètres avec valeurs par défauts :

- Pour chaque fonction suivante écrire deux appels de fonction, le premier en utilisant les valeurs par défauts des paramètres et le deuxième sans l'utiliser. Récupérer et afficher la valeur retournée par chaque appel.

a) `def f4(x=2):`
 `return x`

b) `def f5(a, b=0):`
 `add = a + b`
 `return add`

c) `def f6(a, b=1, msg="addition"):`
 `print(msg)`
 `return a + b`

2. Appels de fonctions avec des arguments nommés :

Remarque : Les arguments peuvent être spécifiés par leur nom, dans n'importe quel ordre.

2.1 Qu'affiche les codes suivants :

```
a) def f7(x, y):  
    return x - y  
  
res1 = f7(x=2, y=3)  
res2 = f7(y=3, x=2)  
print(res1, res2)  
  
b) def f8(a=1, b=2):  
    return a**b  
  
res1 = f8(a=2, b=3)  
res2 = f8(b=3, a=2)  
print(res1, res2)
```

3. Portée d'une variable : globale, locale

Remarque : Les variables locales sont définies à l'intérieur d'une fonction et ne sont pas accessibles en dehors de cette fonction. Les variables globales sont définies à l'extérieur d'une fonction. On peut accéder en lecture à une variable globale depuis n'importe où dans le code directement. Par contre pour affecter une nouvelle valeur à une variable globale depuis une fonction, il faut utiliser le mot clé **global** suivi du nom de la variable en début de fonction, sinon une variable locale du même nom est créée.

3.1 Qu'affiche les codes suivants :

```
a) # lecture de variable globale depuis une fonction  
def f():  
    print(variable_globale)  
  
variable_globale = 10  
f()  
  
b) # modification d'une variable globale depuis une fonction  
def f2():  
    global v2  
    v2 = 5  
  
v2 = 10  
f2()  
print(v2)
```

```

c)def foo():
    L.append(5)

L = [1,2,3]
foo()
print(L)

```

Ecrire du code :

4. Exercices 4 : Fonctions avec paramètres

4.1 Ecrire une fonction nommée `affiche_age` qui prend deux paramètres nommés `nom` et `age`.

- La fonction affiche les paramètres comme dans l'exemple suivant :
Ex : `nom = "Marc", age = 20`
`"Marc : 20 ans"`
- Faire un appel de la fonction avec 2 arguments (`nom : str` et `age : int`).

4.2 Ecrire une fonction nommée `puissance` qui prend deux paramètres `x` et `y`.

- La fonction retourne le résultat de calcul x^y .
- Faire un appel de cette fonction avec deux valeurs entières comme arguments, récupérer et afficher la valeur retournée par la fonction.
- Faire un appel de fonction avec les arguments nommés `x=4` et `y=6`, récupérer et afficher la valeur retournée par la fonction.

4.3 Ecrire une fonction nommée `plus_petit` qui prend une liste de nombres nommée en paramètre.

- Dans la fonction :
 - Ecrire un code qui stocke dans une variable nommée `min_1` le plus petit élément de la liste. Commencer à initialiser la `min_1` variable avec le premier élément de la liste.
 - La fonction retourne la valeur de la variable `min_1`.
- En dehors de la fonction
 - Définir une liste nommée `nombres` contenant des nombres.
 - Faire un appel avec la liste `nombres` en argument, récupérer et afficher la valeur retournée par la fonction.

5. Exercices 5 : Fonctions et paramètres avec valeurs par défauts

Ecrire une fonction nommée `aire` qui prendre un paramètre `r` avec la valeur par défaut de 1.

- Dans la fonction écrire un code qui stocke l'aire d'un cercle dans une variable nommée `a` et il le retourne.
- Faire un appel de fonction sans arguments, récupérer et afficher la valeur retournée par la fonction.
- Faire un appel de fonction avec un argument, récupérer et afficher la valeur retournée par la fonction.